```
function mainLoop(): never
{
    while (shallWeContinue()) {
        print 'Hey ho, on we go...' . PHP_EOL;
    }
}

mainLoop();

Output:
Type something to continue, or press Enter on its own to abort
Uncaught TypeError: mainLoop(): never-returning function must not
implicitly return
```

Now when we get an empty string from user input, the `while` loop is broken, the end of `mainLoop()` is reached, and havoc ensues with a suitably fatal error message. Happily, there is a simple fix:

```
function mainLoop(): never
{
    while (shallWeContinue()) {
        print 'Hey ho, on we go...' . PHP_EOL;
    }

    exit();
}
```

Instead of an implicit return, we have a nice explicit `exit()`, and the program functions as before. It should be obvious that having our `mainLoop()` behavior controlled by the side effect of another function is bad design; `mainLoop()` should have full control over its own behavior. `exit()` (and its alias function `die()`) are fully-fledged functions in PHP 8.4 – with a return type of `never` and accepting a single `int` or `string` parameter but prior to that existed as language constructs with special handling, although special handling still exists in PHP 8.4 to permit the continued use of `exit`/`die` without any parentheses.

By using the `never` return type, we have stated our intentions about how `mainLoop()` should behave. It only took some tiny changes to break our code in a very obvious way. Without the return type declaration, this code would have continued to function without